

SILKY LAB

Project Plan

Mattia Brozzoni

Ivan Caccamo

Simone Preda

Indice

1. Introduzione
2. Modello di processo
3. Organizzazione del progetto
4. Standard, linee guida e procedure
5. Attività di gestione
6. Rischi
7. Personale
8. Metodi e tecniche
9. Garanzia di qualità
10. Workpackages
11. Risorse
12. Budget e programma
13. (Possibili) Cambiamenti
14. Consegna

1. Introduzione

Il progetto Silky Lab nasce con l'obiettivo di creare un e-commerce dedicato all'abbigliamento, che si distingue per l'attenzione alla qualità dei prodotti e un'esperienza utente semplice e intuitiva. L'interfaccia utente rappresenta un aspetto centrale dell'esperienza di Silky Lab, garantendo un design chiaro, minimale e facilmente navigabile, in linea con la filosofia del brand. I capi offerti rifletteranno gli stessi concetti: semplicità e qualità.

2. Modello di processo

Per il progetto abbiamo scelto di adottare Extreme Programming (XP), un metodo agile che si focalizza su pratiche semplici ma efficaci per garantire un sistema sempre funzionante. Lavoreremo in modo incrementale, con integrazioni continue del codice, così da costruire il software passo dopo passo, senza mai perdere di vista la qualità. Ci impegneremo a seguire tutte le linee guida di XP, concentrandoci su tre aspetti fondamentali: condivisione del codice, ritmo sostenibile e programmazione in coppia. Le metodologie di XP ci permetteranno di rimanere agili,

flessibili e sempre pronti ad adattarci, per sviluppare un prodotto solido e rispondere al meglio alle esigenze del progetto.

3. Organizzazione del progetto

Abbiamo scelto di organizzare il nostro team di sviluppo seguendo i principi dell'Agile, mettendo al centro le persone e la collaborazione. Essendo un gruppo ristretto (3 persone), preferiamo una comunicazione diretta e informale. Prediligiamo gli incontri in presenza, lavorando insieme nella stessa stanza, perché crediamo che il contatto diretto favorisca l'efficienza e rafforzi lo spirito di squadra. Non abbiamo ruoli fissi né gerarchie, perché siamo convinti che la qualità sia una responsabilità condivisa da tutti. Per organizzare le attività, utilizzeremo un approccio semplice e funzionale, suddividendo il lavoro in modo che ciascuno possa concentrarsi su ciò che sa fare meglio.

4. Standard, linee guida e procedure

Per lo sviluppo del software gestionale è stato scelto il linguaggio Java, seguendo le linee guida Oracle per garantire un codice chiaro, di qualità e mantenibile. Lavoreremo su Eclipse, un ambiente di sviluppo che permette anche di generare automaticamente la documentazione tecnica con JavaDoc. Per visualizzare e comprendere meglio la struttura del sistema, utilizziamo i diagrammi UML, mentre la gestione del codice e la collaborazione tra sviluppatori avvengono tramite GitHub, una piattaforma pratica per tracciare le modifiche e condividere documentazione. L'interfaccia grafica, invece, verrà sviluppata utilizzando Vaadin, un framework pensato per sviluppare applicazioni moderne e user-friendly.

5. Attività di gestione

Per gestire al meglio il progetto di sviluppo software, abbiamo definito alcune attività che ci permetteranno di lavorare in modo organizzato e di raggiungere gli obiettivi prefissati.

Abbiamo pianificato circa tre incontri settimanali per discutere il lavoro svolto, valutare i risultati, affrontare eventuali difficoltà e proseguire il progetto. Durante questi momenti, analizzeremo lo stato del progetto, stabiliremo le priorità e pianificheremo i passi successivi.

Per tenere sotto controllo lo sviluppo, utilizzeremo Git per la gestione del codice e GitHub come piattaforma per archiviare il progetto e collaborare in modo efficiente. Questi strumenti ci permetteranno di tracciare le modifiche e avere sempre una visione aggiornata del lavoro del team. Infine, per organizzare le attività e monitorare il flusso di lavoro, useremo una Kanban Board. Questa lavagna digitale ci aiuterà a visualizzare il progresso, suddividendo le attività negli stati Backlog, In Progress, Ready, In Review e Done. Ogni aggiornamento sarà immediato, così da tenere tutti allineati sullo stato del progetto.

6. Rischi

Durante la fase di sviluppo del sistema software, potrebbero emergere i seguenti rischi:

1. **Esperienza utente insoddisfacente:** una navigazione poco intuitiva, tempi di caricamento lunghi, difficoltà nella ricerca dei prodotti o un processo di acquisto complicato possono comportare un alto tasso di abbandono del carrello e una bassa conversione.
2. **Malfunzionamenti del sistema di pagamento:** problemi tecnici del sistema di pagamento possono impedire agli utenti di completare l'acquisto, generando frustrazione e quindi portare alla perdita di opportunità di vendita.
3. **Supporto clienti insufficiente:** un servizio clienti inefficace o lento nella risposta alle richieste potrebbe causare insoddisfazione e ridurre la fidelizzazione dei clienti.
4. **Impatto della concorrenza:** l'ingresso di nuovi concorrenti o cambiamenti nelle politiche di prezzo da parte dei principali attori del settore potrebbe influenzare negativamente le vendite e la redditività dell'e-commerce.
5. **Perdita dei dati:** un guasto dell'hardware si potrebbe causare la perdita dei dati, poiché salvati su un database embedded.
6. **Mancato rispetto delle date di scadenza:** ritardi nello sviluppo potrebbero compromettere la consegna del progetto.

7. Personale

Il team è composto da:

- Brozzoni Mattia
- Caccamo Ivan
- Preda Simone

Ognuno di noi avrà compiti simili, con i ruoli e i compiti che verranno distribuiti in modo flessibile, adattandosi alle esigenze del progetto. Non sarà necessario coinvolgere personale esterno, poiché il gruppo gestirà autonomamente tutte le attività necessarie.

8. Metodi e tecniche

- **Ingegneria dei Requisiti**
 - Brainstorming di Gruppo: Questa tecnica ci permetterà di raccogliere idee e definire le funzionalità fondamentali dell'applicazione.
 - Analisi dei Casi d'Uso: Identificheremo i diversi scenari in cui l'utente interagirà con l'applicazione, come la navigazione dei prodotti, l'aggiunta al carrello e il completamento dell'acquisto.

- Interviste Informali: Condurremo brevi interviste con potenziali utenti (ad esempio amici o familiari) per raccogliere feedback e comprendere meglio le funzionalità desiderate.
- Bilanciamento dei requisiti: Distingueremo i requisiti funzionali e non funzionali del sistema gestendo le priorità di ogni requisito con il modello MoSCoW.
- **Progettazione**
 - Progettazione Orientata agli Oggetti (UML): Utilizzeremo diagrammi UML, come i diagrammi delle classi, di sequenza, di comunicazione, ecc., per rappresentare i principali componenti dell'applicazione e le loro interazioni, ad esempio prodotti, utenti e ordini.
- **Implementazione**
 - Linguaggi e Framework: L'applicazione sarà sviluppata in Java utilizzando l'IDE Eclipse. Per la parte dell'interfaccia grafica utilizzeremo il framework Vaadin, che ci consentirà di creare interfacce utente moderne e responsive in maniera efficiente.
 - Controllo di Versione: Utilizzeremo Git e GitHub per il controllo della versione del codice, favorendo la collaborazione e la revisione tra i membri del team.
 - Metodologia Agile: Adotteremo un approccio di sviluppo Agile, iterativo e incrementale, con brevi cicli di sviluppo e frequenti riunioni di gruppo per adattarci rapidamente a eventuali cambiamenti nei requisiti.
- **Test**
 - Ambiente di Test: Utilizzeremo i nostri computer personali come ambiente di test, simulando scenari di utilizzo realistici per valutare il comportamento dell'applicazione e facendo uso di JUnit, per verificare e validare le milestone del progetto.
 - Procedure di accettazione: Definiremo le procedure di test di accettazione per garantire una valutazione soddisfacente delle funzionalità implementate.
 - Test di Usabilità: Coinvolgeremo persone esterne al gruppo per navigare nell'applicazione, e grazie ai loro feedback miglioreremo l'usabilità e l'interfaccia.

9. Garanzia di qualità

Per garantire i diversi parametri di qualità del software agiremo nei seguenti modi:

- **Correttezza:** Il sistema di gestione degli ordini deve garantire che i prodotti selezionati e pagati vengano correttamente registrati e inoltrati per l'elaborazione.
- **Affidabilità:** Il sistema deve garantire che gli ordini non vadano persi, anche in caso di problemi temporanei come un'inaspettata interruzione della connessione.

- **Integrità:** Devono essere implementati sistemi di autenticazione per proteggere i dati degli utenti.
- **Usabilità:** L'interfaccia utente deve essere chiara e ben organizzata, con menu, filtri e categorie facilmente accessibili.
- **Flessibilità:** Il software sarà suddiviso in moduli indipendenti (es. gestione utenti, carrello, pagamenti). Ogni modulo potrà essere aggiornato o sostituito senza impattare sull'intero sistema.
- **Manutenibilità:** Il codice sarà migliorato periodicamente per eliminare complessità inutili o ridondanze, mantenendo alta la qualità e riducendo il rischio di errori futuri.

10. Workpackages

Il progetto sarà suddiviso nei seguenti principali pacchetti di lavoro:

- stesura del project plan e gestione del progetto
- analisi dei requisiti
- design del software
- implementazione
- testing
- manutenzione

Le diverse fasi del progetto saranno articolate in attività specifiche, che verranno distribuite tra i singoli membri del team in base ai rispettivi ruoli e competenze e gestite attraverso la Kanban Board.

11. Risorse

Strumenti per la programmazione e lo sviluppo: il progetto farà uso delle seguenti piattaforme e applicazioni software:

- **Maven:** un sistema per la gestione di progetti Java e l'automazione delle build.
- **Eclipse IDE:** un ambiente di sviluppo integrato e multiplatforma, utilizzato per scrivere codice Java.
- **Vaadin:** un framework open source per lo sviluppo di interfacce utente web, utilizzato per impostare rapidamente l'aspetto dell'applicazione.
- **Papyrus:** un tool open source progettato per creare i diagrammi UML.
- **SQLite:** una libreria software che fornisce un database embedded, per gestire i dati dell'applicazione.
- **JUnit:** un framework open-source per il testing di unità in Java.

Team e risorse umane: Il team di sviluppo, adottando le metodologie di Extreme Programming (XP), si organizzerà suddividendosi e collaborando nei ruoli distinti tra sviluppatori, analisti e tester.

12. Budget e programma

Il budget di riferimento prevede un impegno di 40 ore per ogni membro del team. Particolare attenzione verrà messa sulle fasi di documentazione, modellazione e progettazione che impiegheranno circa il 70% del tempo totale a disposizione. Il restante tempo sarà dedicato alla codifica: design, logica e database. Non è presente un budget monetario.

13. (Possibili) Cambiamenti

Poiché XP è un metodo di sviluppo agile, i cambiamenti sono considerati parte naturale del processo di sviluppo software. Saranno quindi affrontati con flessibilità e gestiti attraverso piccoli aggiustamenti ad ogni iterazione. Le modifiche saranno documentate nello storico dei commit e degli issues di GitHub.

14. Consegna

La consegna del progetto avverrà in due fasi distinte:

- Consegna del Project Plan: questa prima consegna dovrà avvenire circa un mese prima del primo esame scritto, previsto per gennaio. Pertanto, la scadenza è fissata entro il mese di dicembre.
- Consegna del Progetto: questa seconda fase ha termine fissato a cinque giorni prima dell'esame orale.

Metodi di consegna:

- Repository GitHub: il progetto dovrà essere condiviso con il professore Gargantini e la professoressa Bonfanti tramite la repository GitHub.
- Documentazione: il Project Plan dovrà essere indicato nel file README della repository.
- Consegna del Progetto: sarà necessario creare un issue nella repository, intitolata "Approvazione Progetto", da assegnare al professore per la valutazione.